NAME: LIAMBA MUFAYA

S/N: 202501125

PROGRAMME: BACHELOR OF SCIENCE IN COMPUTER SCIENCE

MARKSIMPORT.JAVA

```
import java.io.*;
import java.util.*;
import java.util.logging.*;

// Custom exception
class DuplicateStudentException extends Exception {
    public DuplicateStudentException(String message) {
        super(message);
    }
}

public class MarksImporter {

    private static final Logger logger =
            Logger.getLogger(MarksImporter.class.getName());

    public static void importMarks(String filename) throws IOException {

        Set<String> seen = new HashSet<>();

        // Try-with-resources automatically closes the reader,
        // even if an exception occurs.
        try (BufferedReader br =
                     new BufferedReader(new FileReader(filename))) {

            String line;

            while ((line = br.readLine()) != null) {

                if (line.trim().isEmpty()) {
                    continue;
                }

                String[] parts = line.split(",", -1);

                try {
                    // TODO 1: Check exactly two fields
```

```
                if (parts.length != 2) {
                    throw new IllegalArgumentException(
                            "Expected exactly 2 fields");
                }

                String studentId = parts[0].trim();
                String markStr = parts[1].trim();

                if (studentId.isEmpty()) {
                    throw new IllegalArgumentException(
                            "Student ID cannot be empty");
                }

                // TODO 2: Parse and validate mark
                int mark = Integer.parseInt(markStr);

                if (mark < 0 || mark > 100) {
                    throw new IllegalArgumentException(
                            "Mark out of range (0..100): " + mark);
                }

                // TODO 3: Detect duplicate student IDs
                if (!seen.add(studentId)) {
                    throw new DuplicateStudentException(
                            "Duplicate student ID: " + studentId);
                }

                // TODO 4: Display accepted record
                System.out.println(
                        "Accepted Record -> Student ID: "
                                + studentId + ", Mark: " + mark);

            } catch (IllegalArgumentException |
                     DuplicateStudentException e) {

                // Log technical details for debugging
                logger.log(Level.WARNING,
                        "Skipping invalid record. Exception type: "
                                + e.getClass().getSimpleName()
                                + ", Technical cause: " + e.getMessage());

                // Friendly message for ordinary users
                System.out.println(
                        "Invalid record skipped: " + e.getMessage());
            }
```

```
            }
        }
    }

    // MAIN METHOD
    public static void main(String[] args) {

        try {
            importMarks("marks.txt");

            System.out.println("Import completed successfully.");

        } catch (FileNotFoundException e) {

            logger.log(Level.SEVERE,
                    "File not found: " + e.getMessage(), e);

            System.out.println(
                    "Marks file not found; check the file name.");

        } catch (IOException e) {

            logger.log(Level.SEVERE,
                    "File reading error: " + e.getMessage(), e);

            System.out.println(
                    "An error occurred while reading the marks file.");

        } finally {

            System.out.println("Import attempt finished");
        }
    }
}
```

MARKS.TXT

ICT001, 78

ICT002, ABC

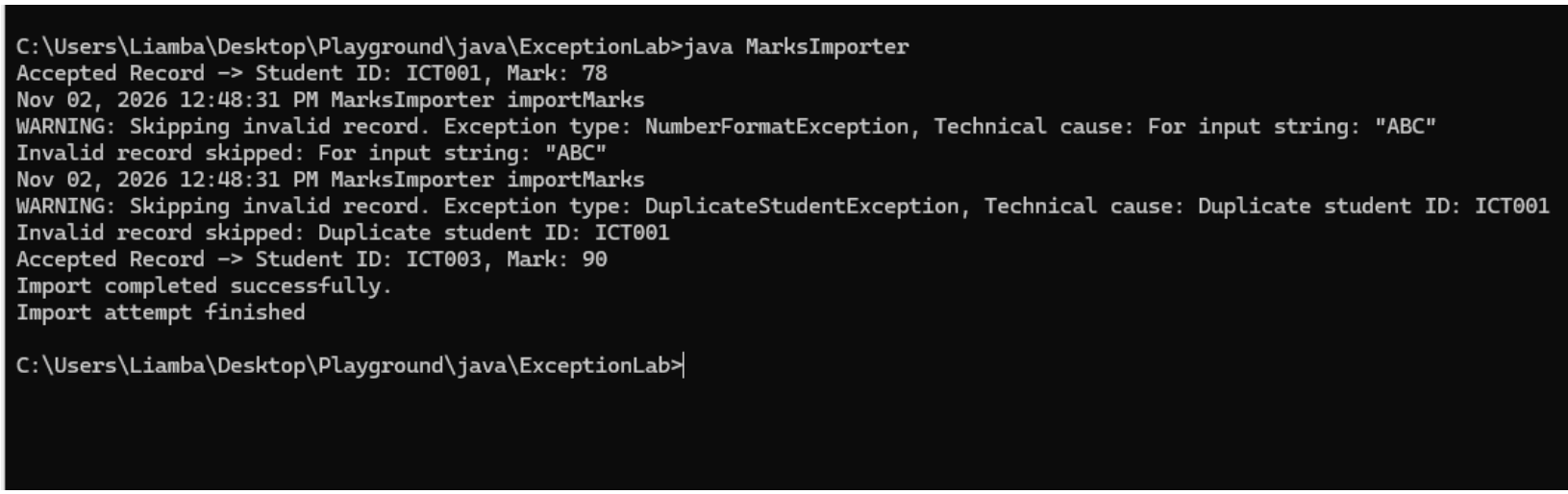
ICT001, 65

ICT003, 90

ACTIVITY A: Predict and Explain

1. **Integer.parseInt("ABC");**
   - **Exception:** NumberFormatException
   - **Classification:** Unchecked (Runtime Exception)
   - **Prevention/Handling Strategy:** Validate the string input to ensure it represents a valid integer before parsing, or wrap the statement in a try-catch block to handle invalid inputs gracefully.
2. **String name = null; name.length();**
   - **Exception:** NullPointerException
   - **Classification:** Unchecked (Runtime Exception)
   - **Prevention/Handling Strategy:** Ensure the object is properly initialized before use, or add a conditional check (if (name != null)) prior to calling methods on it.
3. **new java.io.FileReader("missing.txt");**
   - **Exception:** FileNotFoundException
   - **Classification:** Checked Exception
   - **Prevention/Handling Strategy:** Handle the exception using a try-catch block or declare throws FileNotFoundException in the method signature.

CONSOLE OUTPUTS for T1–T5



- throw is used to explicitly raise an exception. Example: throw new DuplicateStudentException("Duplicate student ID");
- throws declares exceptions that a method may pass to its caller. Example: public static void importMarks(String filename) throws IOException.

3 . Why should the user receive a friendly message while the logger keeps technical details

Friendly messages help users understand the problem without confusing technical information. The logger stores technical details to help programmers identify and fix errors.

4. What would go wrong with an empty catch block, or with catching Exception before FileNotFoundException?

An empty catch block hides errors, making debugging difficult. Catching Exception before FileNotFoundException prevents the specific exception from being caught because FileNotFoundException is a subclass of Exception. This makes the specific catch block unreachable.